

OPERATING SYSTEMS – CS23431

EXPERIMENT 01

1.(A) First come first serve Scheduling Algorithm

First-Come-First-Served CPU Scheduling Algorithm

Write a C program to implement the First-Come-First-Served (FCFS) CPU Scheduling Algorithm. The program should take the number of processes, their Arrival Times (AT), and Burst Times (BT) as input. Calculate and display the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process, along with the Average Turnaround Time and Average Waiting Time.

To evaluate the performance of the algorithm, we use the following formulas:

- **Completion Time (CT)**: The time at which a process finishes execution.
- **Turnaround Time (TAT)**: Total time taken from arrival to completion.
- $TAT = CT - AT$
- **Waiting Time (WT)**: The time a process spends waiting in the ready queue.
- $WT = TAT - BT$

Input Format

1. The first input line contains an integer representing the number of processes.
2. The next lines contain two integers for each process, where the first integer represents the Arrival Time (AT) and the second integer represents the Burst Time (BT).

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

```
3
0 2
1 2
2 3
```

Sample Output 1

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 1.00

```
1 #include <iostream.h>
2 int main()
3 {
4     int n;
5     printf("Enter number of processes: \n");
6     scanf("%d", &n);
7     int at[n], bt[n], ct[n], tat[n], wt[n];
8     float avg_tat, avg_wt;
9     for(int i=0; i<n; i++)
10     {
11         printf("Enter Arrival Time and Burst Time for P%d: \n", i+1);
12         scanf("%d %d", &at[i], &bt[i]);
13     }
14     ct[0]=at[0] + bt[0];
15     for(int i=1; i<n; i++)
16     {
17         int temp=ct[i-1] + at[i];
18         ct[i]=temp+bt[i];
19     }
20     for(int i=0; i<n; i++)
21     {
22         tat[i]=ct[i] - at[i];
23         wt[i]=tat[i] - bt[i];
24     }
25     for(int i=0; i<n; i++)
26     {
27         avg_tat+=tat[i];
28         avg_wt+=wt[i];
29     }
30     printf("\nAverage Turnaround Time: %.2f\n", avg_tat/n);
31     printf("Average Waiting Time: %.2f\n", avg_wt/n);
32     return 0;
33 }
```

Test Cases

Custom Test Case

Sample Test Case

Test Case - 1

Input:

```
3
0 2
1 2
2 3
```

Expected Output:

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:

Average Turnaround Time: 3.33
Average Waiting Time: 1.00

FIGURE: 1.1

1.(b) Shortest Job First Scheduling Algorithm

Non-Preemptive Shortest Job First CPU Scheduling Algorithm

Write a C program to implement the Non-Preemptive Shortest Job First (SJF) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), and Burst Times (BT). At any given time, the process with the minimum burst time among the arrived processes must be executed first. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and find the overall average TAT and WT.

Core Logic

Selection: Out of all processes that have arrived by the current time, the one with the minimum Burst Time is chosen.

Formulas:

- $Turnaround\ Time\ (TAT) = Completion\ Time\ (CT) - Arrival\ Time\ (AT)$
- $Waiting\ Time\ (WT) = Turnaround\ Time\ (TAT) - Burst\ Time\ (BT)$

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The next lines contain two integers for each process, where the first integer denotes the Arrival Time (AT) of the process and the second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

```
4
0 8
0 8
0 7
0 3
```

Sample Output 1

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00
Average Waiting Time = 7.00

```
1 #include <iostream.h>
2 int main()
3 {
4     int n;
5     printf("Enter number of processes: \n");
6     scanf("%d", &n);
7     int at[n], bt[n], ct[n], tat[n], wt[n];
8     for(int i=0; i<n; i++)
9     {
10         printf("Enter Arrival Time and Burst Time for P%d: \n", i+1);
11         scanf("%d %d", &at[i], &bt[i]);
12     }
13     int finish[0];
14     for(int i=0; i<n; i++)
15     {
16         finish[i]=0;
17     }
18     int count=0;
19     int time=0;
20     while(count<n)
21     {
22         int min=999;
23         int index=-1;
24         for(int i=0; i<n; i++)
25         {
26             if(time<=at[i] && bt[i]<min && finish[i]==0)
27             {
28                 min=bt[i];
29                 index=i;
30             }
31             if(index==-1)
32             {
33                 continue;
34             }
35             ct[index]=time+bt[index];
36             tat[index]=ct[index]-at[index];
37             wt[index]=tat[index]-bt[index];
38             finish[index]=1;
39             count++;
40             time=ct[index];
41         }
42     }
```

Test Cases

Custom Test Case

Sample Test Case

Test Case - 1

Input:

```
4
0 8
0 8
0 7
0 3
```

Expected Output:

Enter number of processes:
Enter Arrival Time and Burst Time for P1:
Enter Arrival Time and Burst Time for P2:
Enter Arrival Time and Burst Time for P3:
Enter Arrival Time and Burst Time for P4:

Average Turnaround Time = 13.00
Average Waiting Time = 7.00

Output:

FIGURE: 1.2

1.(c) Non-Preemptive Priority Scheduling Algorithm

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Example 1

Sample Input 1

```
3
0 5 2
1 3 1
2 2 3
```

Sample Output 1

```
Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67
Average Waiting Time = 3.33
```

Select Language : C (GCC 9.2.0)

```

1 #include<stdio.h>
2 int main()
3 {
4     printf("Enter number of processes: \n");
5     int n;
6     scanf("%d",&n);
7     int at[n],bt[n],p[n],tt[n],ct[n],wt[n];
8     for(int i=0;i<n;i++)
9     {
10        printf("Enter AT, BT and Priority for P%d: \n",i+1);
11        scanf("%d %d %d",&at[i],&bt[i],&p[i]);
12    }
13    int finish[n];
14    for(int i=0;i<n;i++)
15        finish[i]=0;
16    int time=0;
17    int count=0;
18    float s=0,t=0;
19    while(count<n)
20    {
21        int index=-1;
22        int pr=999;
23        for(int i=0;i<n;i++)
24        {
25            if(finish[i]==0 && p[i]<pr && time==at[i])
26            {
27                pr=p[i];
28                index=i;
29            }
30            if(index!=-1)
31            {
32                time++;
33                continue;
34            }
35            ct[index]=time+bt[index];
36            time=ct[index];
37            tt[index]=ct[index]-at[index];
38            wt[index]=tt[index]-bt[index];
39        }
40        count++;
41    }
42    s=(float)ct[n]/n;
43    t=(float)tt[n]/n;
44    printf("Average Turnaround Time = %.2f\n",s);
45    printf("Average Waiting Time = %.2f\n",t);
46    return 0;
47 }
```

Test Cases 2

[Run Sample Cases](#) [Run All Cases](#)

Custom Test Case

> Custom Test

Sample Test Case

Test Case - 1

Input :

```
3
0 5 2
1 3 1
2 2 3
```

Expected Output :

```

Enter number of processes:
Enter AT, BT and Priority for P1:
Enter AT, BT and Priority for P2:
Enter AT, BT and Priority for P3:

Average Turnaround Time = 6.67
Average Waiting Time = 3.33
```

FIGURE: 1.3

1.(d) Round Robin Scheduling Algorithm

Round Robin CPU Scheduling Algorithm

Write a C program to implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format

- The first line of input contains an integer representing the number of processes.
- The second line contains an integer representing the Time Quantum.
- The next lines contain two integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT) of the process.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Sample Input 1

```
3
2
0 5
1 3
2 1
```

Sample Output 1

```
Enter number of processes:
Enter Time Quantum:
Enter AT and BT for P1:
Enter AT and BT for P2:
Enter AT and BT for P3:

Average Turnaround Time: 6.33
Average Waiting Time: 3.33
```

Select Language : C (GCC 9.2.0)

```

1 #include<stdio.h>
2 int main()
3 {
4     int n;
5     printf("Enter number of processes: \n");
6     scanf("%d",&n);
7     int at[n],bt[n],ct[n],tat[n],wt[n],rm[n];
8     int tq;
9     printf("Enter Time Quantum: \n");
10    scanf("%d",&tq);
11    for(int i=0;i<n;i++)
12    {
13        printf("Enter AT and BT for P%d: \n",i+1);
14        scanf("%d %d",&at[i],&bt[i]);
15    }
16    for(int i=0;i<n;i++)
17    {
18        rm[i]=bt[i];
19    }
20    double avgwt=0,avgat=0;
21    int time=0;
22    int count=0;
23    while(count<n)
24    {
25        for(int i=0;i<n;i++)
26        {
27            if((time==at[i] && rm[i]>0) )
28            {
29                executed=1;
30                if(rm[i]>tq)
31                {
32                    time+=tq;
33                    rm[i]-=tq;
34                }
35                else
36                {
37                    time+=rm[i];
38                    rm[i]=0;
39                    count++;
40                }
41            }
42        }
43    }
44    s=(float)ct[n]/n;
45    t=(float)wt[n]/n;
46    printf("Average Turnaround Time = %.2f\n",s);
47    printf("Average Waiting Time = %.2f\n",t);
48    return 0;
49 }
```

Test Cases 2

[Run Sample Cases](#) [Run All Cases](#)

Custom Test Case

> Custom Test

Sample Test Case

Test Case - 1

Input :

```
3
2
0 5
1 3
2 1
```

Expected Output :

```

Enter number of processes:
Enter Time Quantum:
Enter AT and BT for P1:
Enter AT and BT for P2:
Enter AT and BT for P3:

Average Turnaround Time: 6.33
Average Waiting Time: 3.33
```

FIGURE: 1.4

EXPERIMENT 02

2) Banker's Deadlock Avoidance Algorithm

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

- The first line of input contains an integer representing the number of processes.
- The second line contains an integer representing the number of resource types.
- The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
- This is followed by the Maximum Demand (Max) matrix with the same dimensions.
- The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Output Format

The output displays whether the system is in a safe state or not. If the system is in a safe state, the program prints the Safe Sequence of processes. If the system is not in a safe state, the program indicates that a deadlock may occur.

Example 1

Sample Input 1

```
3
3
0 1 0
2 0 0
3 0 3
3 2 2
2 1 1
4 3 3
0 0 0
```

Sample Input 1

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2 int main()
3 {
4     int n, r;
5     printf("Enter number of processes: \n");
6     scanf("%d", &n);
7     printf("Enter number of resource types: \n");
8     scanf("%d", &r);
9     int alloc[n][r], max[n][r], need[n][r];
10    int avail[r], finish[n], safeSeq[n];
11    printf("Enter Allocation Matrix: \n");
12    for(int i=0; i<n; i++)
13    {
14        for(int j=0; j<r; j++)
15        {
16            scanf("%d", &alloc[i][j]);
17        }
18    }
19    printf("Enter Max Matrix: \n");
20    for(int i=0; i<n; i++)
21    {
22        for(int j=0; j<r; j++)
23        {
24            scanf("%d", &max[i][j]);
25        }
26    }
27    printf("Enter Available Resources: \n");
28    for(int i=0; i<r; i++)
29    {
30        scanf("%d", &avail[i]);
31    }
32    for(int i=0; i<n; i++)
33        finish[i]=0;
34    for(int i=0; i<n; i++)
35    {
36        for(int j=0; j<r; j++)
37            need[i][j]=max[i][j]-alloc[i][j];
38    }
```

Test Cases

Custom Test Case

> Custom Test

Sample Test Case

Test Case - 1

Input:

```
3
3
0 1 0
2 0 0
3 0 3
3 2 2
2 1 1
4 3 3
0 0 0
```

Expected Output:

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```

FIGURE: 2.0

EXPERIMENT 03

3.(a) FIFO Page Replacement Algorithm

FIFO Page Replacement Algorithm

Write a C program to implement the FIFO Page Replacement Algorithm. The program should take the number of frames and a reference string (sequence of page requests) as input. For each page in the reference string, determine if it results in a 'Page Hit' or a 'Page Fault'. Display the state of the frames after each request and calculate the total number of page faults and hits.

Input Format

- The first line of input contains an integer representing the number of pages in the reference string.
- The second line contains the page reference string as a sequence of integers.
- The third line contains an integer representing the number of available frames.

Output Format

The total number of Page Faults and Page Hits are displayed.

Sample Input 1

```
7
1 3 0 3 5 5 3
3
```

Sample Output 1

Enter number of pages in reference string:

Enter the reference string:

Enter number of frames:

Total Page Faults: 6

Total Page Hits: 1

How it works

Ref String	Frames	Status
1	1 - -	Fault
3	1 3 -	Fault
0	1 3 0	Fault

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2 int main()
3 {
4     int n, frames;
5     printf("Enter number of pages in reference string: \n");
6     scanf("%d", &n);
7     printf("Enter the reference string: \n");
8     int ref[n];
9     for(int i=0; i<n; i++)
10    {
11        scanf("%d", &ref[i]);
12    }
13    printf("Enter number of frames: \n");
14    scanf("%d", &frames);
15    int frame[frames];
16    for(int i=0; i < frames; i++)
17    {
18        frame[i]=-1;
19    }
20    int front = 0;
21    int faults = 0;
22    int hits = 0;
23    for(int i=0; i < n; i++)
24    {
25        int found = 0;
26        for(int j=0; j < frames; j++)
27        {
28            if(frame[j] == ref[i])
29            {
30                found = 1;
31                hits++;
32                break;
33            }
34        }
35        if(!found)
36        {
37            frame[front] = ref[i];
38            front = (front + 1) % frames;
39            faults++;
40        }
41    }
```

Test Cases

Custom Test Case

> Custom Test

Sample Test Case

Test Case - 1

Input:

```
7
1 3 0 3 5 5 3
3
```

Expected Output:

```
Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:
Total Page Faults: 6
Total Page Hits: 1
```

FIGURE:

3.(b) LRU Page Replacement Algorithm

LRU Page Replacement Algorithm

Write a C program to implement the LRU Page Replacement Algorithm. The program should accept a reference string and the number of frames. For each page request, identify whether it results in a Hit or a Fault. If a Fault occurs and the frames are full, replace the page that was accessed least recently. Display the frame status at each step and calculate the total number of Page Faults and Hits.

Input Format

- The first line of input contains an integer representing the number of pages in the reference string.
- The second line contains the page reference string as a sequence of integers.
- The third line contains an integer representing the number of available frames.

Output Format

The output displays the total number of Page Faults and the total number of Page Hits after processing the entire reference string using the LRU page replacement algorithm.

Sample Input 1

```
5
1 2 3 4 5
5
```

Sample Output 1

Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 5
Total Page Hits: 0

How it works

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2 int main(){
3     int n, frames;
4     printf("Enter number of pages in reference string: \n");
5     scanf("%d", &n);
6     int ref[n];
7     printf("Enter the reference string: \n");
8     for(int i=0; i < n; i++){
9         scanf("%d", &ref[i]);
10    }
11    printf("Enter number of frames: \n");
12    scanf("%d", &frames);
13    int frame[frames];
14    int time[frames];
15    for(int i=0; i < frames; i++){
16        frame[i] = -1;
17        time[i] = 0;
18    }
19    int faults = 0;
20    int hits = 0;
21    int counter = 0;
22    for(int i=0; i < n; i++){
23        int found = 0;
24        for(int j=0; j < frames; j++){
25            if(frame[j] == ref[i]){
26                hits++;
27                counter++;
28                time[j] = counter;
29                found = 1;
30                break;
31            }
32        }
33        if(!found){
34            int lruIndex = 0;
35            for(int j = 1; j < frames; j++){
36                if(time[j] < time[lruIndex]){
37                    lruIndex = j;
38                }
39            }
40            frame[lruIndex] = ref[i];
41            time[lruIndex] = counter;
42            faults++;
43        }
44    }
45    printf("Total Page Faults: %d\n", faults);
46    printf("Total Page Hits: %d\n", hits);
47    return 0;
48 }
```

Test Cases

Run Sample Cases

Custom Test Case

Custom Test

Sample Test Case

Test Case - 1

Input:

```
5
1 2 3 4 5
5
```

Expected Output:

```
Enter number of pages in reference string:
Enter the reference string:
Enter number of frames:

Total Page Faults: 5
Total Page Hits: 0
```

FIGURE: 3.2

EXPERIMENT 04

4.(a)First Fit Contiguous Memory Allocation

First Fit Contiguous Memory Allocation

Write a program to implement the First Fit contiguous memory allocation technique. The program should take the sizes of various memory blocks and the sizes of processes as input. For each process, search through the memory blocks sequentially and allocate the first block that meets the size requirement. If no such block exists, indicate that the process cannot be allocated. Finally, display the allocation details and calculate internal fragmentation.

Input Format

- The first line contains an integer n representing the number of memory blocks.
- The next line contains n integers representing the sizes of each memory block.
- The next line contains an integer m representing the number of processes.
- The next line contains m integers representing the sizes of each process.

Output Format

For each process, the program prints whether the process is allocated or not. If allocated, it displays the process number, process size, block number, block size, and the internal fragmentation (block size minus process size). If not allocated, it indicates that the process cannot be allocated.

Sample Input 1

```
5
100 200 300 500 600
4
250 420 150 400
```

Sample Output 1

Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:

Process 1 of size 250 is allocated to Block 2 of size 500 with Fragment 200
Process 2 of size 420 is allocated to Block 5 of size 600 with Fragment 180
Process 3 of size 150 is allocated to Block 3 of size 300 with Fragment 150
Process 4 of size 400 is not allocated

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2 int main(){
3     int n, m;
4     printf("Enter number of memory blocks: \n");
5     scanf("%d", &n);
6     int b[n];
7     printf("Enter size of each block: \n");
8     for(int i=0; i < n; i++){
9         scanf("%d", &b[i]);
10    }
11    printf("Enter number of processes: \n");
12    scanf("%d", &m);
13    int p[m];
14    printf("Enter size of each process: \n");
15    for(int i=0; i < m; i++){
16        scanf("%d", &p[i]);
17    }
18    for(int i=0; i < m; i++){
19        int allocated=0;
20        for(int j=0; j < n; j++){
21            if(p[i] <= b[j]){
22                printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d", i+1, p[i], j+1, b[j], b[j]-p[i]);
23                allocated=1;
24                break;
25            }
26        }
27        if(!allocated){
28            printf("Process %d of size %d is not allocated", i+1, p[i]);
29        }
30    }
31    return 0;
32 }
```

Test Cases

Run Sample Cases

Custom Test Case

Custom Test

Sample Test Case

Test Case - 1

Input:

```
5
100 200 300 500 600
4
250 420 150 400
```

Expected Output:

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:

Process 1 of size 250 is allocated to Block 2 of size 500 with Fragment 200
Process 2 of size 420 is allocated to Block 5 of size 600 with Fragment 180
Process 3 of size 150 is allocated to Block 3 of size 300 with Fragment 150
Process 4 of size 400 is not allocated
```

FIGURE: 4.1

4.(b) Best Fit Contiguous Memory Allocation

Best Fit Contiguous Memory Allocation

Write a C program to implement the Best Fit Contiguous Memory Allocation technique. The program should accept the sizes of memory blocks and the sizes of processes as input.

For each process, the program searches all available memory blocks and allocates the smallest block that is sufficient to satisfy the process size. If no suitable block is available, the process is marked as not allocated. The program should display the allocation details for each process and calculate the internal fragmentation.

Input Format

- The first line contains an integer representing the number of memory blocks.
- The second line contains the sizes of the memory blocks.
- The third line contains an integer representing the number of processes.
- The fourth line contains the sizes of the processes.

Output Format

For each process, display whether it is allocated or not.

If allocated, display the block number, block size, and internal fragmentation.

If not allocated, display that the process could not be allocated.

Example 1

Sample Input 1

```
5
500 800 200 300 600
4
232 437 532 409
```

Sample Output 1

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 232 is allocated to Block 4 of size 500 with Fragment 68
Process 2 of size 437 is allocated to Block 3 of size 300 with Fragment 63
Process 3 of size 532 is allocated to Block 2 of size 800 with Fragment 68
Process 4 of size 409 is allocated to Block 5 of size 600 with Fragment 174
```

Example 2

Sample Input 2

```
3
500 800 300
```

```
1 //include <stdio.h>
2 int main()
3 {
4     printf("Enter number of memory blocks:\n");
5     scanf("%d",&n);
6     int b[n];
7     int p[n];
8     printf("Enter size of each block:\n");
9     for(int i=0;i<n;i++)
10         scanf("%d",&b[i]);
11 }
12
13 printf("Enter number of processes:\n");
14 scanf("%d",&m);
15 int d[m];
16 printf("Enter size of each process:\n");
17 for(int i=0;i<m;i++)
18     scanf("%d",&d[i]);
19
20 for(int i=0;i<m;i++)
21 {
22     int best=-1;
23     for(int j=0;j<n;j++)
24     {
25         if(b[j]>=d[i])
26             best=j;
27     }
28     if(best!=-1)
29     {
30         printf("Process %d of size %d is allocated to Block %d of size %d with Fragment %d\n",i+1,d[i],best+1,b[best],b[best]-d[i]);
31         b[best]-=d[i];
32     }
33     else
34     {
35         printf("Process %d of size %d is not allocated\n",i+1,d[i]);
36     }
37 }
```

Test Cases

Custom Test Case

Sample Test Case

Test Case - 1

Input :

```
5
500 800 200 300 600
4
232 437 532 409
```

Expected Output :

```
Enter number of memory blocks:
Enter size of each block:
Enter number of processes:
Enter size of each process:
Process 1 of size 232 is allocated to Block 4 of size 500 with Fragment 68
Process 2 of size 437 is allocated to Block 3 of size 300 with Fragment 63
Process 3 of size 532 is allocated to Block 2 of size 800 with Fragment 68
Process 4 of size 409 is allocated to Block 5 of size 600 with Fragment 174
```

FIGURE: 4.2